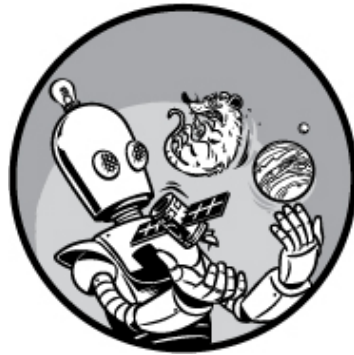


10

ARE WE ALONE? EXPLORING THE FERMI PARADOX



Scientists use the *Drake equation* to estimate the possible number of civilizations in the galaxy currently producing electromagnetic emissions, such as radio waves. In 2017, the equation was updated to account for new exoplanet discoveries by NASA's Kepler satellite. The result, published in the scientific journal *Astrobiology*, was astonishing. For humanity to be the first and only technologically advanced species, the probability of an advanced civilization developing on a habitable alien planet would have to be less than 1 in 10 billion trillion! And yet, as Nobel Prize-winning physicist Enrico Fermi famously observed, “Where is everybody?”

Fermi was more skeptical about interstellar travel than the existence of extraterrestrials, but his question became known as *Fermi's paradox*, and it morphed into the conjecture “If they were out there, they'd be here.” According to the SETI Institute, even with modest rocket technology, an eager civilization could explore the entire galaxy, if not colonize it, within 10 million years. That may sound like a long time, but it's only 1/1,000 the age of the Milky Way! As a result, some have come to accept Fermi's paradox as proof we are alone in the cosmos. Others find holes in the argument.

In this chapter, you'll investigate the absence of alien radio transmissions by calculating the probability of one civilization detecting another based on the volume of their transmissions and output from the Drake equation. You'll also use Python's de facto standard GUI package, `tkinter`, to quickly and easily create a graphical model of the Milky Way.

Project #17: Modeling the Milky Way

Our galaxy, the Milky Way, is a fairly common spiral galaxy, like the one shown in **Figure 10-1**.



Figure 10-1: Spiral galaxy NGC 6744, “Big Brother” to the Milky Way

In cross-sectional view, the Milky Way is a flattened disc with a central bulge that most likely contains a supermassive black hole at its core. Four “spiral arms”—comprising relatively densely packed gas, dust, and stars—radiate from this central mass. The dimensions of the Milky Way are shown in **Figure 10-2**.

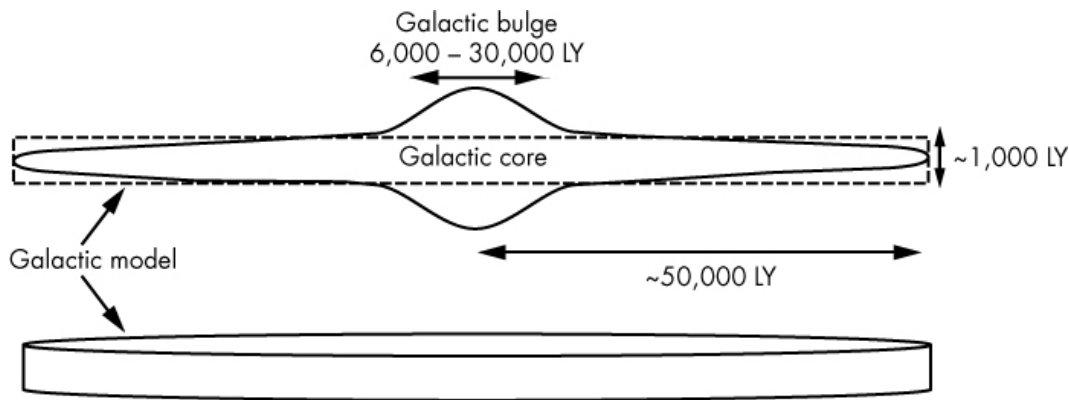


Figure 10-2: Schematic profile (edge view) of the Milky Way galaxy (LY = Light-Years) and a simplified model

The center of the galaxy is considered fairly inhospitable to life due to high levels of radiation associated with the more closely packed stars. So, for this project, you can treat the galaxy as a simple disc to discount some of the volume associated with the bulge but still leave room for some advanced civilizations near the core (see the Galactic Model in **Figure 10-2**).

THE OBJECTIVE

For a given number of advanced galactic civilizations and an average radio bubble size, estimate the probability of *any* civilization detecting the radio transmissions of *any other* civilization. For perspective, post the size of Earth's current radio bubble on a 2D graphical representation of the Milky Way.

The Strategy

Here are the steps needed to complete this project:

1. Estimate the number of transmitting civilizations using the Drake equation.
2. Choose a size range for their radio bubbles.
3. Generate a formula for estimating the probability of one civilization detecting another.
4. Build a graphical model of the galaxy and post Earth's radio emissions bubble.

In order to keep the description close to the code, each of these tasks will be described in detail in its own section. Note that the first two steps don't require the use of Python.

Estimating the Number of Civilizations

You can manually estimate the number of advanced civilizations using the Drake equation:

$$N = R^* \cdot f_p \cdot n_e \cdot f_l \cdot f_i \cdot f_c \cdot L$$

where:

N = The number of civilizations in our galaxy whose electromagnetic emissions are detectable

R^* = The average rate of star formation in the galaxy (new stars per year)

f_p = The fraction of stars with planets

n_e = For stars with planets, the average number of planets with an environment suitable for life

f_l = The fraction of planets that develop life

f_i = The fraction of life-bearing planets with intelligent, civilized life

f_c = The fraction of civilizations that release detectable signs of their existence into space

L = The length of time—in years—over which the civilizations release the detectable signals

Thanks to recent advances in the detection of exoplanets, the first three components (R^* , f_p , n_e) are becoming increasingly constrained. For n_e , recent studies suggest that 10 to 40 percent of all planets may be suitable for *some form* of life.

For the remaining components, Earth is the only example. In the Earth's 4.5-billion-year history, *Homo sapiens* has existed for only 200,000 years,

civilization for only 6,000 years, and radio transmissions for only 112 years. With respect to L , wars, plagues, ice ages, asteroid impacts, super-volcanoes, supernovas, and coronal mass ejections can disrupt a civilization's ability to transmit radio signals. And the shorter the time of transmission, the less likely it is for civilizations to coexist.

According to the Wikipedia article on the Drake equation (https://en.wikipedia.org/wiki/Drake_equation), in 1961, Drake and his colleagues estimated the number of communicating civilizations in the galaxy to be between 1,000 and 100,000,000. More recent updates have set the range from 1 (just us) to 15,600,000 (**Table 10-1**).

Table 10-1: Some Drake Equation Inputs and Results

Parameter	Drake 1961**	Drake 2017	Your choices
R^*	1	3	
f_p	0.35	1	
n^e	3	0.2	
f_i	1	0.13	
f_i	1	1	
f_c	0.15	0.2	
L	50×10^6	1×10^9	
N	7.9×10^6	15.6×10^6	

**midpoint of ranges shown

For input to your program, you can use the estimates in the table, those you find online, or those you calculate yourself (in the final column of the table).

Selecting Radio Bubble Dimensions

Radio waves that aren't focused into a beam for targeted transmission are incidental. Think of these as "planet leakage." Because we choose not to broadcast our presence to aliens who might come and eat us, almost all of our transmissions are incidental. These transmissions currently form an expanding sphere around Earth with a diameter of around 225 light-years (LY).

A 225 LY bubble sounds impressive, but it is the *detectable* size that really matters. A radio wave front is subject to the *inverse square law*, which means it continuously loses power density as it expands. Additional power loss can result from absorption or scattering. At some point, the signal becomes too weak to separate from background noise. Even with our best technology—the radio telescopes of the *Breakthrough Listen* program—we could detect our own radio bubble only out to about 16 LY.

Since we're really investigating why *we* haven't detected aliens, you should assume, for this project, that other civilizations have technology similar to our own. Another assumption should be that, like us, all aliens have a paranoid planetary consciousness and aren't broadcasting "here we are" signals that would announce their presence. Investigating incidental bubble sizes ranging from a little smaller than those currently detectable to those a little larger than our own transmissions should be a reasonable place to start. This would suggest a diameter range of 30 to 250 LY. Although we can't detect a 250 LY bubble, it will be interesting to see what the odds would be if we could.

Generating a Formula for the Probability of Detection

As the number of advanced civilizations in the galaxy increases, the probability that one will detect another also increases. This is intuitive, but how do you assign the actual probabilities?

The nice thing about computers is that they allow us to brute-force our way to solutions that may or may not be intuitive. One approach here would be to make a 3D model of the Milky Way disc, randomly distribute civilizations throughout, and measure the distances between them using one of Python's many tools for calculating Euclidian distance. But with

potentially hundreds of millions of civilizations to analyze, this method would be computationally expensive.

Since we're dealing with huge unknowns, there's no need to be super-accurate or precise. We just want to be in the ballpark, so an easy simplification is to compartmentalize the galaxy into a series of radio bubble "equivalent volumes" by dividing the volume of the galactic disc by the volume of a radio bubble (see **Figure 10-3**).

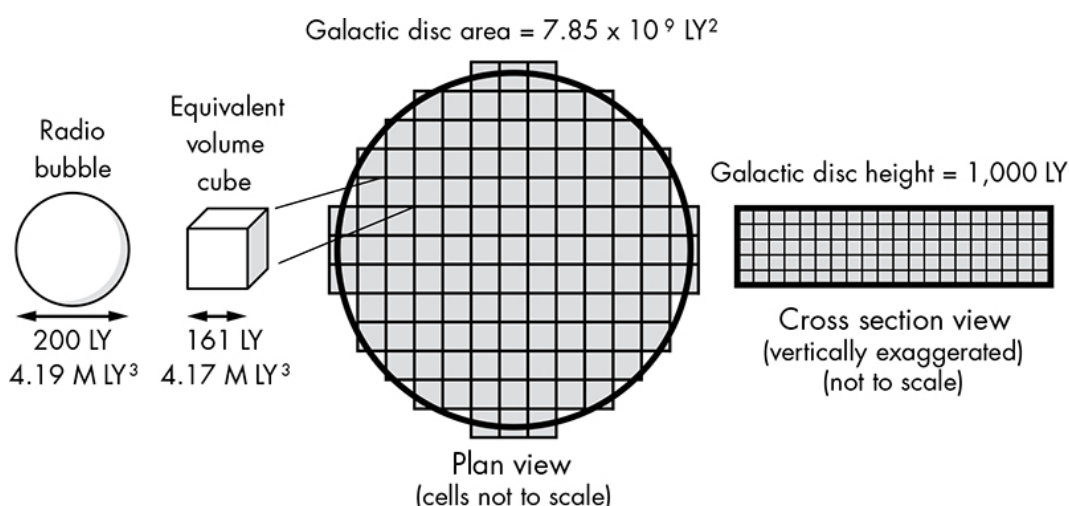


Figure 10-3: Modeling the galaxy using cubes with volumes equivalent to a 200 LY radio bubble

You can find the volumes using the following equations, where R is the radius of the galactic disc and r is the radius of a radio bubble:

$$\text{disc volume} = \pi \times R^2 \times \text{disc height}$$

$$\text{radio bubble volume} = \frac{4}{3} \times \pi \times r^3$$

$$\text{scaled disc volume} = \text{disc volume} / \text{radio bubble volume}$$

The scaled disc volume is the number of equivalent volumes that “fit” in the galaxy. Think of these as boxes numbered from 1 to the maximum number of volumes.

To place civilizations, you simply choose a box number at random. Duplicate picks indicate multiple civilizations within the same box. Assume civilizations in the same box can detect each other. This isn't strictly true (see **Figure 10-4**), but because you'll be using large numbers of civilizations, the discrepancies will tend to cancel each other out, just as when you sum a lot of rounded numbers.

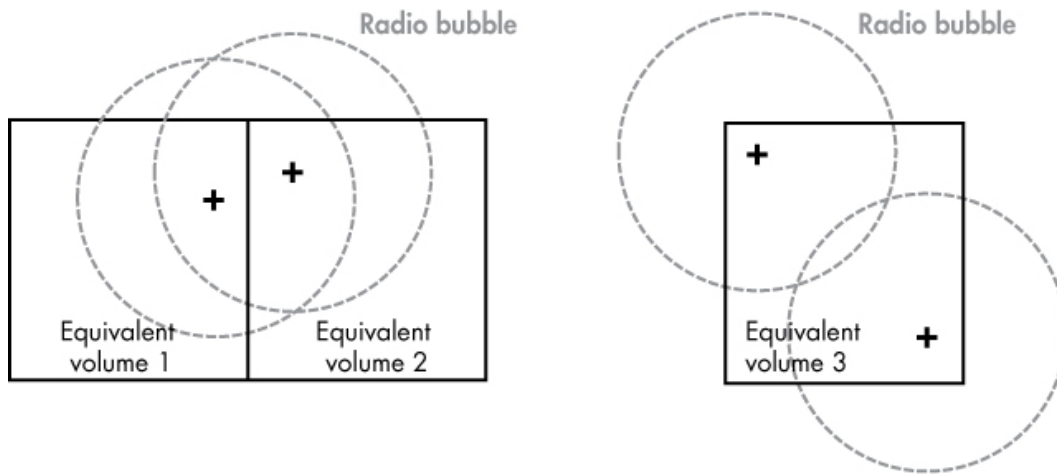


Figure 10-4: Detection problems at the individual equivalent volume level

To avoid having to repeat this exercise every time you change the number of civilizations and/or radio bubble dimensions, you can capture the results as a formula—a *polynomial equation*—that can be used to generate all future probability estimates. A *polynomial* is the sum or difference of a collection of algebraic terms. The famous *quadratic equation* we all learned in school is a polynomial equation of the second degree (meaning that the exponents of the variables are no greater than 2):

$$ax^2 + bx + c = 0$$

Polynomials make nice curves, so they're tailor-made for this problem. But for the formula to work with variable numbers of civilizations and bubble sizes, you'll need to use the *ratio* of the number of civilizations to the total volume. The total volume is represented by the scaled disc volume, which is the same as the total number of equivalent volumes.

In **Figure 10-5**, each dot represents the probability of detection for the ratio below it. The equation shown in the figure is the polynomial expression, which generates the line connecting the dots. With this formula, you can predict the probability for any ratio of civilizations per volume, up to a value of 5 (above this, we'll just assume the probability is 1.0).

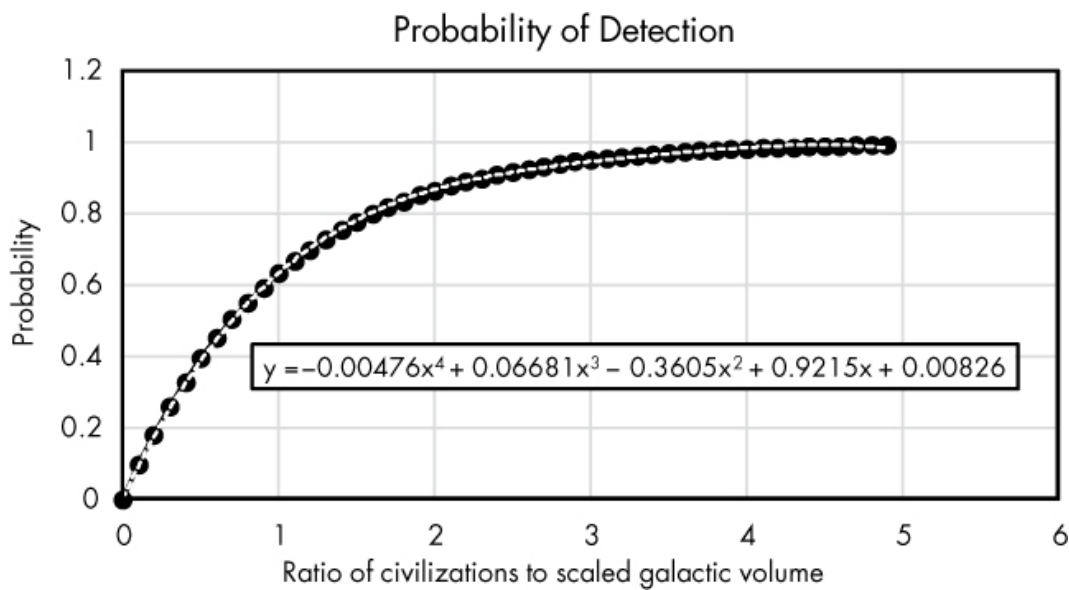


Figure 10-5: Probability of detection versus ratio of number of civilizations to scaled galactic volume

In **Figure 10-5**, the civilizations-to-volume ratios are posted on the x-axis. A ratio of 0.5, for example, means that there are half as many civilizations as available radio bubble equivalent volumes, a ratio of 2 means there are twice as many civilizations as volumes, and so on. The y-axis is the probability that an equivalent volume contains more than one civilization.

Another thing to note from **Figure 10-5** is that it takes a lot of civilizations to ensure that they all have a roommate. Imagine that 999,999 out of 1,000,000 equivalent volumes contain at least two civilizations and you use your God-like powers to randomly place one new civilization. The odds are one in a million that this new civilization will end up in the remaining volume with a single occupant. That last equivalent volume is the proverbial needle in the haystack!

NOTE

*An axiom of computer modeling is to start simple and add complexity. The simplest “base case” assumption is that advanced civilizations are randomly distributed throughout the galaxy. In “**Challenge Projects**” on **page 214**, you’ll get a chance to challenge this assumption using the concept of galactic habitable zones.*

The Probability-of-Detection Code

The probability-of-detection code randomly chooses locations (radio bubble equivalent volumes) for a set number of locations and civilizations, counts how many locations occur only once (that is, contain only one civilization), and repeats the experiment multiple times to converge on a probability estimate. The process is then repeated for a new number of civilizations. The output is presented as probability versus the *ratio* of civilizations per volume, rather than the actual number of civilizations, and turned into a polynomial expression so that the results are easily portable. This means this program only needs to be run once.

To generate the polynomial equation and check that it fits the data, you'll use `NumPy` and `matplotlib`. The `NumPy` library adds support for large, multidimensional arrays and matrices, and it includes many mathematical functions that operate on them. The `matplotlib` library supports 2D plotting and rudimentary 3D plotting, and `NumPy` represents its numerical mathematics extension.

There are several ways to install these scientific Python distributions. One way is to use `SciPy`, an open source Python library used for scientific and technical computing (see <https://scipy.org/index.html>). If you're going to do a lot of data analysis and plotting, you may want to download and use a free package like Anaconda or Enthought Canopy, which works with Windows, Linux, and macOS. These packages spare you the task of finding and installing all the required data science libraries in the correct version. A listing of these types of packages, along with links to their websites, can be found at <https://scipy.org/install.html>.

Alternatively, you may want to download the products directly, using `pip`. I did this using the instructions at <https://scipy.org/install.html>. Because `matplotlib` requires a large number of dependencies, these need to be installed at the same time. For Windows, I ran the following Python 3-specific command from the PowerShell, launched from within my `Python35` folder (you can leave off the 3 in `python3` unless you have multiple versions installed):

```
$ python3 -m pip install --user numpy scipy matplotlib ipython jupyter pandas sympy nose
```

All the other modules you'll need come bundled with Python. As for the code in **Listings 10-1** and **10-2**, you can type it in or download a copy from <https://www.nostarch.com/impracticalpython/>.

Calculating Probability of Detection for a Range of Civilizations

Listing 10-1 imports modules and does all the work just described except for fitting the polynomial and displaying the `matplotlib` quality check.

probability_of_detection.py, part 1

```
❶ from random import randint
    from collections import Counter
    import numpy as np
    import matplotlib.pyplot as plt

❷ NUM_EQUIV_VOLUMES = 1000 # number of locations in which to place civilization
    MAX_CIVS = 5000 # maximum number of advanced civilizations
    TRIALS = 1000 # number of times to model a given number of civilizations
    CIV_STEP_SIZE = 100 # civilizations count step size

❸ x = [] # x values for polynomial fit
    y = [] # y values for polynomial fit

❹ for num_civs in range(2, MAX_CIVS + 2, CIV_STEP_SIZE):
    civs_per_vol = num_civs / NUM_EQUIV_VOLUMES
    num_single_civs = 0
    ❺ for trial in range(TRIALS):
        locations = [] # equivalent volumes containing a civilization
        ❻ while len(locations) < num_civs:
            location = randint(1, NUM_EQUIV_VOLUMES)
            locations.append(location)
        ❼ overlap_count = Counter(locations)
            overlap_rollup = Counter(overlap_count.values())
            num_single_civs += overlap_rollup[1]

❽ prob = 1 - (num_single_civs / (num_civs * TRIALS))
```

```
# print ratio of civs-per-volume vs. probability of 2+ civs per location
❸ print("{:.4f} {:.4f}".format(civs_per_vol, prob))
❿ x.append(civs_per_vol)
    y.append(prob)
```

Listing 10-1: Imports modules, randomly chooses radio-bubble-equivalent-volume locations, and calculates the probability of multiple civilizations per location

Import the familiar `random` module and `Counter`, for counting the number of civilizations at each location (designated by how many times a location has been chosen) ❶. How `Counter` works will be explained in a moment. You'll use the `NumPy` and `matplotlib` imports to fit and display the polynomial.

Assign some constants that represent user input for the number of equivalent volumes, the maximum number of civilizations, the number of trials—that is, how many times to repeat the experiment for a given number of civilizations—and a step size for the count ❷. Because the results are predictable, you can use a large step value of 100 without compromising accuracy. Note that you'll get very similar results whether the number of equivalent volumes is 100 or 100,000+.

You'll need a series of paired (x, y) values for the polynomial expression, so start two lists to hold these ❸. The x -value will be the ratio of civilizations per volume, and the y -value will be the corresponding probability of detection.

Start a series of nested loops, with the highest loop representing the number of civilizations to model ❹. You need at least two civilizations for one to detect the other, and set the maximum to `MAX_CIVS` plus 2, to overshoot when calculating the polynomial. Use the `CIV_STEP_SIZE` constant for the step value.

Next, calculate the overall `civs_per_vol` ratio and start a counter named `num_single_civs` to keep track of the number of locations containing a single civilization.

You've chosen how many civilizations to distribute, so now use a `for` loop to go through the number of trials ⑤. For each trial, you distribute the same number of civilizations. Assign an empty list to the variable `locations` and then, for each civilization ⑥, pick a location number at random and append it to the list. Duplicate values in the list will represent locations containing multiple civilizations.

Run `Counter` on this list ⑦ and get the values. End the loop by getting the number of locations that occur only once and add them to the `num_single_civs` counter. Here's an example of how these three statements work:

```
>>> from collections import Counter
>>> alist = [124, 452, 838, 124, 301]
>>> count = Counter(alist)
>>> count
Counter({124: 2, 452: 1, 301: 1, 838: 1})
>>> value_count = Counter(count.values())
>>> value_count
Counter({1: 3, 2: 1})
>>> value_count[1]
3
```

The `alist` list contains five numbers, with one (124) duplicated. Running `Counter` on this list produces a dictionary with the numbers as the keys and the number of times they occur as the values. Passing `Counter` the values in `count`—with the `values()` method—creates another dictionary with the previous values as keys and the number of times they occur as the new values. You want to know how many numbers occur only once, so use the dictionary method `value_count[1]` to return the number of nonduplicated numbers. These, of course, would represent radio-bubble-equivalent volumes containing a single civilization.

Now use the results from `Counter` to calculate the probability of multiple civilizations per location for the current number of civilizations being distributed ⑧. This is 1 minus the number of single-occupancy locations

divided by the number of civilizations in each trial times the number of trials.

Follow this by printing the ratio of civilizations to volume and the probability that multiple civilizations share a location ⑨. The first few lines of this output are as follows:

```
0.0020  0.0020
0.1020  0.0970
0.2020  0.1832
0.3020  0.2607
0.4020  0.3305
0.5020  0.3951
0.6020  0.4516
0.7020  0.5041
```

This printout serves as an initial QC step and is optional; comment it out if you want to speed up the runtime. Finish by appending the values to the `x` and `y` lists ⑩.

Generating a Predictive Formula and Checking the Results

Listing 10-2 uses NumPy to perform polynomial regression on the probability of detection versus the ratio of civilizations per volume calculated in **Listing 10-1**. You'll use this polynomial equation in the next program to obtain probability estimates. To check that the resulting curve fits the data points, matplotlib displays the actual and predicted values.

probability_of_detection.py, part 2

```
① coefficients = np.polyfit(x, y, 4) # 4th order polynomial fit
② p = np.poly1d(coefficients)
   print("\n{}".format(p))
③ xp = np.linspace(0, 5)
④ _ = plt.plot(x, y, '.', xp, p(xp), '-')
```

```
⑤ plt.ylim(-0.5, 1.5)
⑥ plt.show()
```

Listing 10-2: Performs polynomial regression and displays a QC plot

Start by assigning a variable, `coefficients`, to the output from the NumPy `polyfit()` method ❶. This method takes as arguments the `x` and `y` lists and an integer representing the degree of the fitting polynomial. It returns a vector of coefficients, `p`, that minimizes the squared error.

If you print the `coefficients` variable, you get the following output:

```
[ -0.00475677  0.066811  -0.3605069  0.92146096  0.0082604 ]
```

To get the full expression, pass the `coefficients` variable to `poly1d` and assign the results to a new variable ❷. Print this variable, and you'll see a similar equation to that shown in **Figure 10-5**:

```
          4          3          2
-0.004757 x + 0.06681 x - 0.3605 x + 0.9215 x + 0.00826
```

To check that the polynomial adequately reproduces the input, you'll want to plot the ratio of civilizations to volume on the x-axis, with probability on the y-axis. To get the x-axis values, you can use the NumPy `linspace()` method, which returns evenly spaced numbers over a specified interval. Use a range of `(0, 5)`, as this will cover almost the full probability range.

To post symbols for the calculated and predicted values, first pass the `plot()` method the `x` and `y` lists, plotting them using a period (`.`), which is equivalent to the dots in **Figure 10-5** ❸. Then pass the predicted x-axis values (`xp`) and, to get the predicted y-axis probability, pass `p` the same variable, plotting the results using a dash.

Finish by limiting the y-axis to values of `-0.5` and `1.5` ❹ and use the `show()` method to actually display the graph (**Figure 10-6**) ❺. The resultant plot is simple and sparse, as its only purpose is to confirm that the polynomial

regression is working as intended. You can alter the polynomial fit by increasing or decreasing the third argument in step ❶.

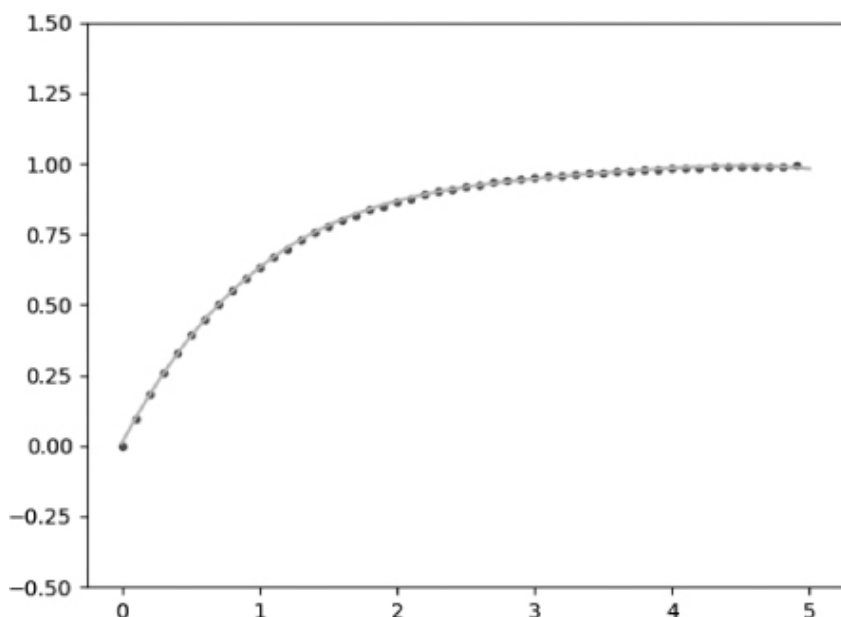


Figure 10-6: Calculated results (dots) versus results predicted by the polynomial (line)

Armed with these results, you can now estimate the probability of detection for any number of civilizations in the blink of an eye. All Python needs to do is solve a polynomial equation.

Building the Graphical Model

The graphical model will be a 2D, top-down view of the galactic disc. Plotting the size of Earth’s present emissions bubble on this display will put in perspective both the size of the galaxy and our diminutive place within it.

Modeling the Milky Way is all about modeling the spiral arms. Each spiral arm represents a *logarithmic spiral*, a geometric feature so common in nature it has been dubbed *spira mirabilis*—“miraculous spiral.” If you compare **Figure 10-7** to **Figure 10-1**, you can see how closely the structure of a hurricane resembles that of a galaxy. The eye of the hurricane can even be thought of as a supermassive black hole, with the eyewall representing the event horizon!

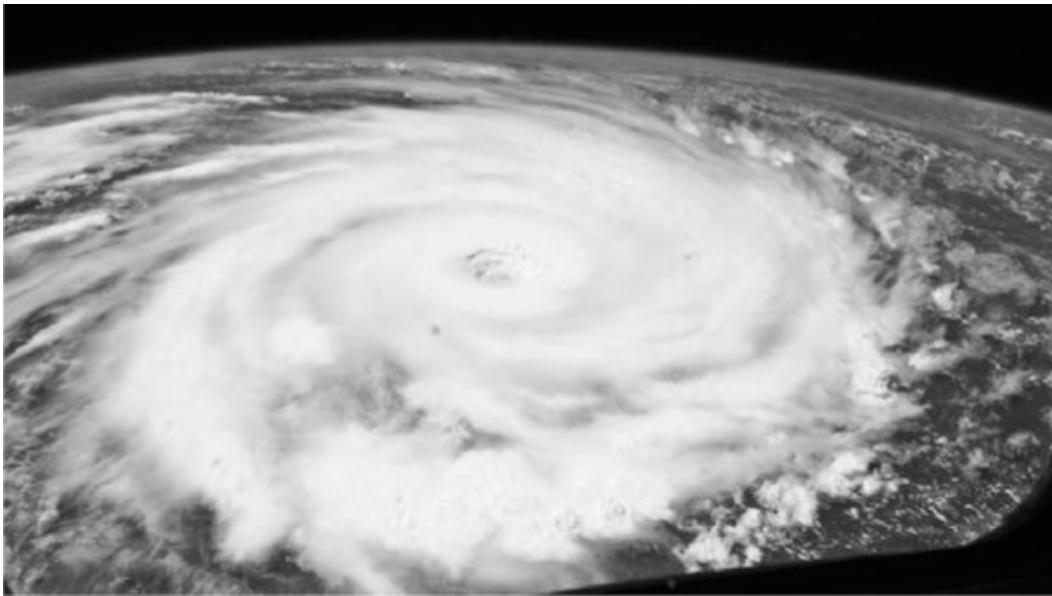


Figure 10-7: Hurricane Igor

Because spirals radiate out from a central point, or *pole*, you'll more easily graph them with *polar coordinates* (Figure 10-8). With polar coordinates, the (x, y) coordinates used in the more familiar Cartesian coordinate system are replaced by (r, θ) , where r is the distance from the center and θ is the angle made by r and the x-axis. The coordinates for the pole are $(0, 0)$.

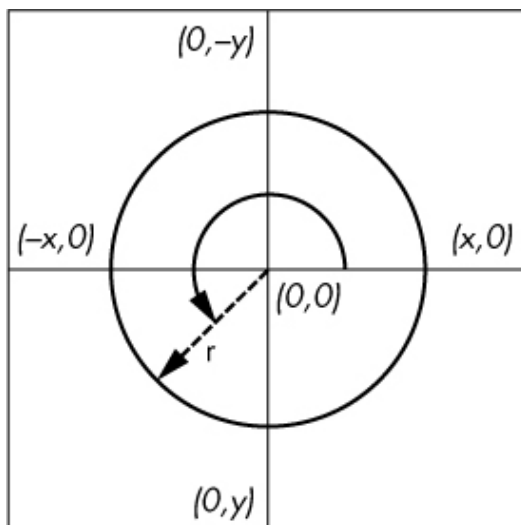


Figure 10-8: Example polar coordinate system

The polar equation for a logarithmic spiral is:

$$r = ae^{b\theta}$$

where r is the distance from the origin, θ is the angle from the x-axis, e is the base of natural logarithms, and a and b are arbitrary constants.

You can use this formula to draw a single spiral; then, rotate and redraw the spiral three times to produce the four arms of the Milky Way. You'll build the spirals out of circles of various sizes, which will represent stars.

Figure 10-9 is an example of one realization of the graphical model.

Because the simulations are stochastic, each will be slightly different, and there are multiple variables you can tweak to change the appearance.

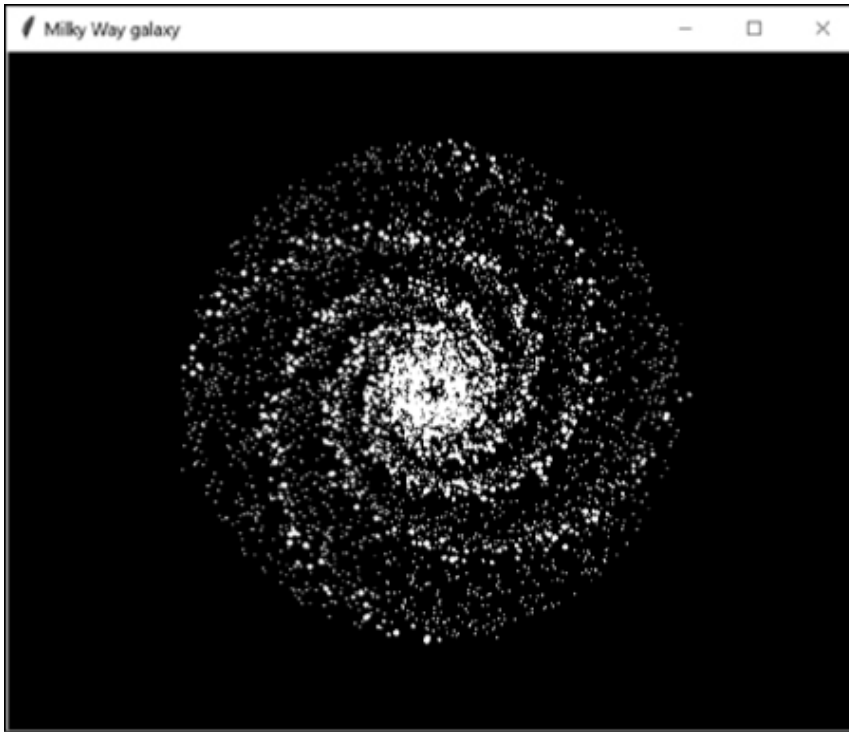


Figure 10-9: The Milky Way, modeled using logarithmic spirals

I generated the image in **Figure 10-9** with `tkinter` (pronounced “tee-kay-in-ter”), the default GUI library for developing desktop applications in Python. Although primarily designed for GUI elements such as windows, buttons, scroll bars, and so on, `tkinter` can also generate graphs, charts, screensavers, simple games, and more. Among its advantages is that, as part of the standard Python distribution, it’s portable across all operating systems and there’s no need to install external libraries. It’s also well documented and easy to use.

Most Windows, macOS, and Linux machines come with `tkinter` already installed. If you don’t have it or need the latest version, you can download and install it from <https://www.activestate.com/>. As always, if the module is already installed, you should be able to import it in the interpreter window with no errors:

```
>>> import tkinter  
>>>
```

Introductory Python books sometimes include overviews of `tkinter`, and you can find the official online documentation at <https://docs.python.org/3/library/tk.html>. Some other references on `tkinter` are included in “**Further Reading**” on **page 212**.

Scaling the Graphical Model

The scale of the graphical model is in light-years per pixel, and the width of each pixel will equate to the diameter of a radio bubble. As a result, when the radio bubble being investigated changes diameter, the scale units will change, and the graphical model will need to be rebuilt. The following equation will scale the model to the bubble:

$$\text{scaled disc radius} = \text{disc radius} / \text{bubble diameter}$$

where the disc radius is 50,000 and the length unit is light-years.

When the selected radio bubble is small, the graphical model “zooms in,” and when it is large, it “zooms out” (**Figure 10-10**).

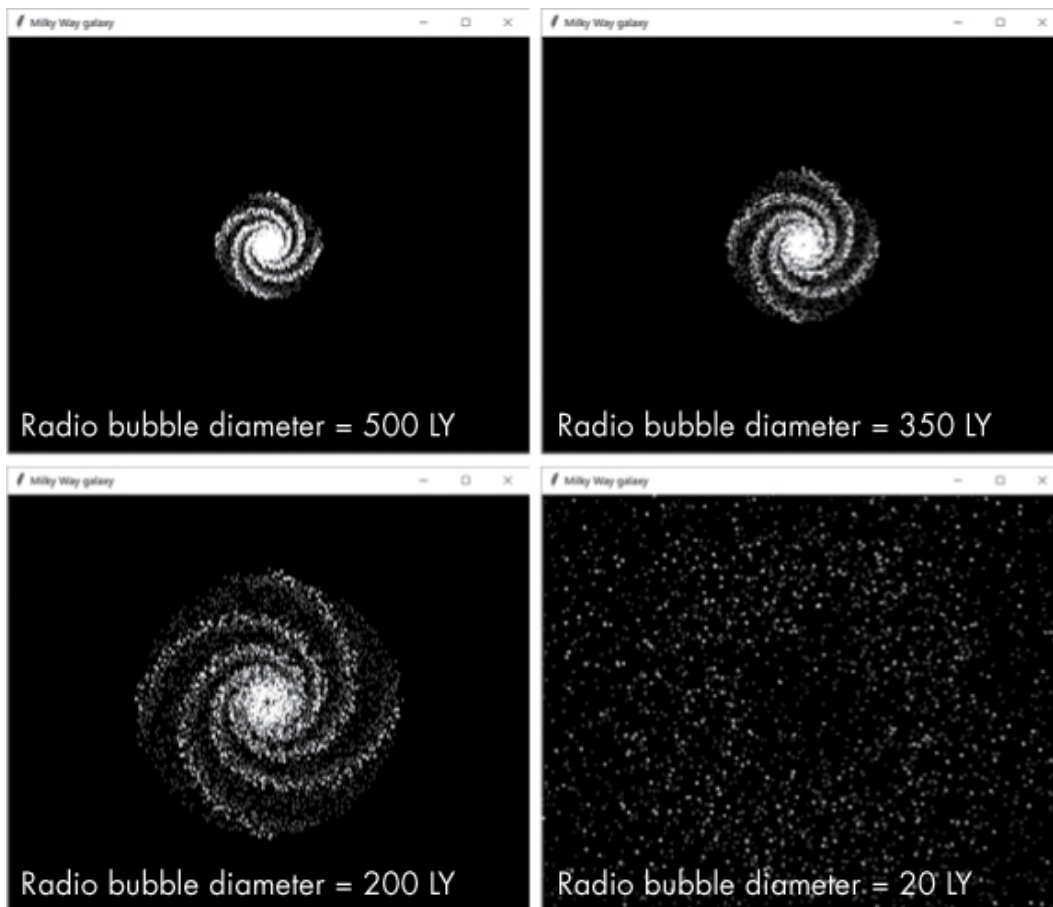


Figure 10-10: Impact of radio bubble diameter on the appearance of the galactic model

The Galaxy Simulator Code

The galaxy simulator code will calculate the probability of detection for any number of civilizations and radio bubble sizes, and then generate a graphical model of the galaxy. When a bubble the size of our current emissions bubble is used, it will post and annotate our bubble in red, in the approximate location of our solar system. You can download the code from <https://www.nostarch.com/impracticalpython/>.

Entering Inputs and Key Parameters

Listing 10-3 starts *galaxy_simulator.py* by importing modules and putting the frequently accessed user input near the top.

galaxy_simulator.py, part 1

```
❶ import tkinter as tk
    from random import randint, uniform, random
```

```

import math

#=====

❷ # MAIN INPUT

# scale (radio bubble diameter) in light-years:
❸ SCALE = 225 # enter 225 to see Earth's radio bubble

# number of advanced civilizations from the Drake equation:
❹ NUM_CIVS = 15600000

#=====

```

Listing 10-3: Imports modules and assigns constants

Import `tkinter` as `tk`, so you don't have to type the full name when calling `tkinter` classes ❶. If you're using Python 2, use `Tkinter`—with an uppercase *T*. You'll also need the `random` and `math` modules.

Use a comment to highlight the main user-input section ❷ and assign the two inputs. Use `SCALE` for the diameter, in light-years, of the detectable electromagnetic bubble around each civilization ❸; `NUM_CIVS` is the number of civilizations to model, which you can determine using anything from the Drake equation to a total guess ❹.

Setting Up the `tkinter` Canvas and Assigning Constants

The code in **Listing 10-4** instantiates a `tkinter` window object with a canvas on which you can draw things. This is where the galaxy map, or graphical model, will appear. It also assigns constants related to the dimensions of the Milky Way.

galaxy_simulator.py, part 2

```

# set up display canvas
❶ root = tk.Tk()
    root.title("Milky Way galaxy")
❷ c = tk.Canvas(root, width=1000, height=800, bg='black')
❸ c.grid()

```

```
④ c.configure(scrollregion=(-500, -400, 500, 400))

    # actual Milky Way dimensions (light-years)
⑤ DISC_RADIUS = 50000
    DISC_HEIGHT = 1000
⑥ DISC_VOL = math.pi * DISC_RADIUS**2 * DISC_HEIGHT
```

Listing 10-4: Sets up `tkinter` window and canvas and assigns constants

Start by creating a window with the conventional name `root` ❶. This is a *top-level* window that will hold everything else. In the next line, give the window a title—“Milky Way galaxy”—which will appear in the upper left of the window frame (see **Figure 10-9** for an example).

Next, add a component, known as a *widget*, to the root window. *Widget* stands for “Windows gadget.” There are 21 core widgets in `tkinter`, including labels, frames, radio buttons, and scroll bars. Assign a `Canvas` widget to contain all the drawing objects ❷. This is a general-purpose widget intended for graphics and other complex layouts. Specify the parent window, the screen width and height, and the background color. Name the canvas `c`, for *canvas*.

You can divide the `Canvas` widget into rows and columns, like a table or spreadsheet. Each cell in this grid can hold a different widget, and these widgets can span multiple cells. Within a cell, you can align a widget using the `STICKY` option. To manage each widget in a window, you’ll need to use the `grid` geometry manager. Since you’re using only one widget in this project, you don’t need to pass the manager anything ❸.

Finish by configuring the `canvas` to use a `scrollregion` ❹. This sets the origin coordinates `(0, 0)` to the center of the `canvas`. You need this to draw the galaxy’s spiral arms with polar coordinates. Without it, the default origin would be the top-left corner of the `canvas`.

The arguments passed to `configure` set the limits of the `canvas`. These should be half of the `canvas` width and height; for example, scroll limits of `600, 500` will require `canvas` dimensions of `1200, 1000`. The values shown here work

well on a small laptop, but feel free to alter them later if you find you need a larger window.

Follow the input section with dimensional constants for the galaxy ⑤. You could assign some of these variables within functions, but having them in the global space allows for a more logical flow to the code explanation.

The first two are the radius and height of the galactic disc from **Figure 10-2**. The final constant represents the disc volume ⑥.

Scaling the Galaxy and Calculating the Probability of Detection

Listing 10-5 defines functions to scale the galaxy dimensions based on the diameter of the radio bubble in use and to calculate the probability of one civilization detecting another. The latter function is where you apply the polynomial equation built with the *probability_of_detection.py* program described previously.

galaxy_simulator.py, part 3

```
① def scale_galaxy():
    """Scale galaxy dimensions based on radio bubble size (scale)."""
    disc_radius_scaled = round(DISC_RADIUS / SCALE)
    ② bubble_vol = 4/3 * math.pi * (SCALE / 2)**3
    ③ disc_vol_scaled = DISC_VOL/bubble_vol
    ④ return disc_radius_scaled, disc_vol_scaled

⑤ def detect_prob(disc_vol_scaled):
    """Calculate probability of galactic civilizations detecting each other."
    ⑥ ratio = NUM_CIVS / disc_vol_scaled # ratio of civs to scaled galaxy volur
    ⑦ if ratio < 0.002: # set very low ratios to probability of 0
        detection_prob = 0
        elif ratio >= 5: # set high ratios to probability of 1
            detection_prob = 1
    ⑧ else:
        detection_prob = -0.004757 * ratio**4 + 0.06681 * ratio**3 - 0.3605 *
```



```
ratio**2 + 0.9215 * ratio + 0.00826
9 return round(detection_prob, 3)
```

Listing 10-5: Scales galactic dimensions and calculates detection probability

Define a function called `scale_galaxy()` to scale the galactic dimensions to the radio bubble size ❶. It will use the constants from the global space, so there is no need to pass it any arguments. Calculate the scaled disc radius and then the radio bubble volume, using the equation for the volume of a sphere, and assign the results to `bubble_vol` ❷.

Next, divide the actual disc volume by `bubble_vol` to get the scaled disc volume ❸. This is the number of radio bubble “equivalent volumes” that can fit in the galaxy. Each bubble constitutes a possible location for a civilization.

End the function by returning the `disc_radius_scaled` and `disc_vol_scaled` variables ❹.

Now, define a function called `detect_prob()` to calculate the probability of detection, which takes the scaled disc volume as an argument ❺. For the x term in the polynomial, calculate the ratio of the number of civilizations to the scaled disc volume ❻. Since the polynomial regression can have problems at the endpoints, use conditionals to set very small ratios to 0 and large ratios to 1 ❼. Otherwise, apply the polynomial expression generated by the *probability_of_detection.py* code ❽, then return the probability rounded to three decimal places ❾.

Using Polar Coordinates

Listing 10-6 defines a function to select random (x, y) locations using polar coordinates. This function will choose the locations of some of the stars posted in the graphical model. Because the display is 2D, there’s no need to choose a z location.

galaxy_simulator.py, part 4

```

❶ def random_polar_coordinates(disc_radius_scaled):
    """Generate uniform random (x, y) point within a disc for 2D display."""
    ❷ r = random()
    ❸ theta = uniform(0, 2 * math.pi)
    ❹ x = round(math.sqrt(r) * math.cos(theta) * disc_radius_scaled)
        y = round(math.sqrt(r) * math.sin(theta) * disc_radius_scaled)
    ❺ return x, y

```

Listing 10-6: Defines a function to randomly pick an (x, y) pair with polar coordinates

The function takes the scaled disc radius as an argument ❶. Use the `random()` function to choose a float value between 0.0 and 1.0 and assign it to the variable `r` ❷. Next, randomly choose `theta` from a uniform distribution between 0 and 360 degrees (2π is the radian equivalent of 360 degrees) ❸.

The transformation to generate points *evenly* over a *unit* disc is:

$$x = \sqrt{r} * \cos \theta$$

$$y = \sqrt{r} * \sin \theta$$

The equations yield (x, y) values between -1 and 1. To scale the results to the galactic disc, multiply by the scaled disc radius ❹. End the function by returning `x` and `y` ❺.

Building Spiral Arms

Listing 10-7 defines a function that builds the spiral arms using the logarithmic spiral equation. This spiral may be miraculous, but a large part of the magic is tinkering with the initial bare-bones spiral to flesh out the arm. You'll accomplish this by varying the size of stars, randomly altering their positions a tiny amount, and duplicating the spiral for each arm in order to shift it slightly backward and dim its stars.

galaxy_simulator.py, part 5

```

❶ def spirals(b, r, rot_fac, fuz_fac, arm):
    """Build spiral arms for tkinter display using logarithmic spiral formula

```

```

    b = arbitrary constant in logarithmic spiral equation
    r = scaled galactic disc radius
    rot_fac = rotation factor
    fuzz_fac = random shift in star position in arm, applied to 'fuzz' variable
    arm = spiral arm (0 = main arm, 1 = trailing stars)
    """

    ❷ spiral_stars = []
    ❸ fuzz = int(0.030 * abs(r)) # randomly shift star locations
        theta_max_degrees = 520
    ❹ for i in range(theta_max_degrees): # range(0, 600, 2) for no black hole
        theta = math.radians(i)
        x = r * math.exp(b * theta) * math.cos(theta + math.pi * rot_fac)\
            + randint(-fuzz, fuzz) * fuzz_fac
        y = r * math.exp(b * theta) * math.sin(theta + math.pi * rot_fac)\
            + randint(-fuzz, fuzz) * fuzz_fac
        spiral_stars.append((x, y))
    ❺ for x, y in spiral_stars:
        ❻ if arm == 0 and int(x % 2) == 0:
            c.create_oval(x-2, y-2, x+2, y+2, fill='white', outline='')
        elif arm == 0 and int(x % 2) != 0:
            c.create_oval(x-1, y-1, x+1, y+1, fill='white', outline='')
        ❼ elif arm == 1:
            c.create_oval(x, y, x, y, fill='white', outline='')

```

Listing 10-7: Defines the `spirals()` function

Define a function called `spirals()` ❶. Its parameters are listed in the function docstring. The first two parameters, `b` and `r`, are from the logarithmic spiral equation. The next, `rot_fac`, is the rotation factor that lets you move the spiral around the center point so you can produce a new spiral arm. The fuzz factor, `fuzz_fac`, lets you tweak how far you move stars away from the center of the spiraling line. Finally, the `arm` parameter lets you specify either the leading arm or the trailing arm of faint stars. The trailing arm will be shifted—that is, plotted a little behind the leading arm—and its stars will be smaller.

Initialize an empty list to hold the locations of the stars that will make up the spiral ❷. Assign a `fuzz` variable, where you multiply an arbitrary constant by the absolute value of the scaled disc radius ❸. The spiral equation alone produces stars that are lined up (see the left two panels in **Figure 10-11**). Fuzzing will move stars in the spiral back and forth a little, to either side of the spiral line. You can see the effect on the bright stars in the rightmost panel in **Figure 10-11**. I determined these values through trial and error; feel free to play with them if you like.

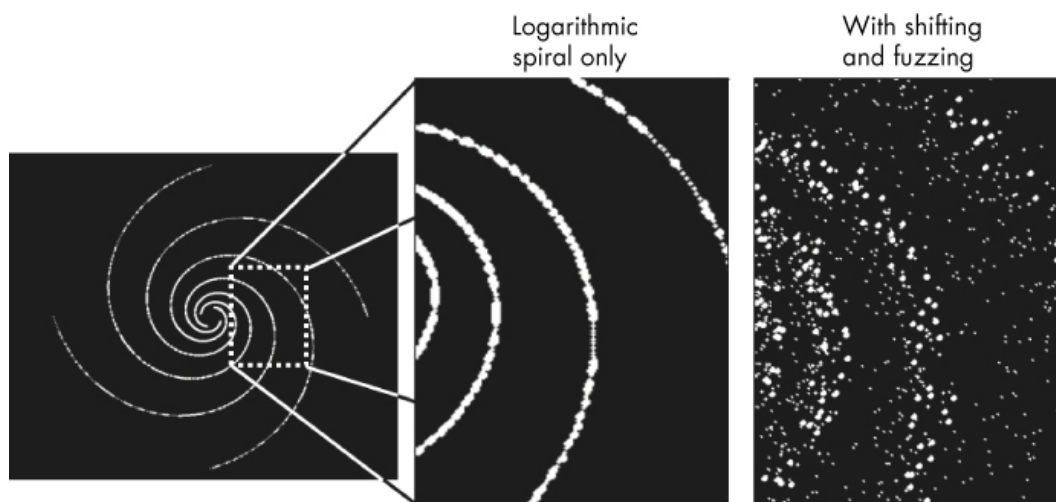
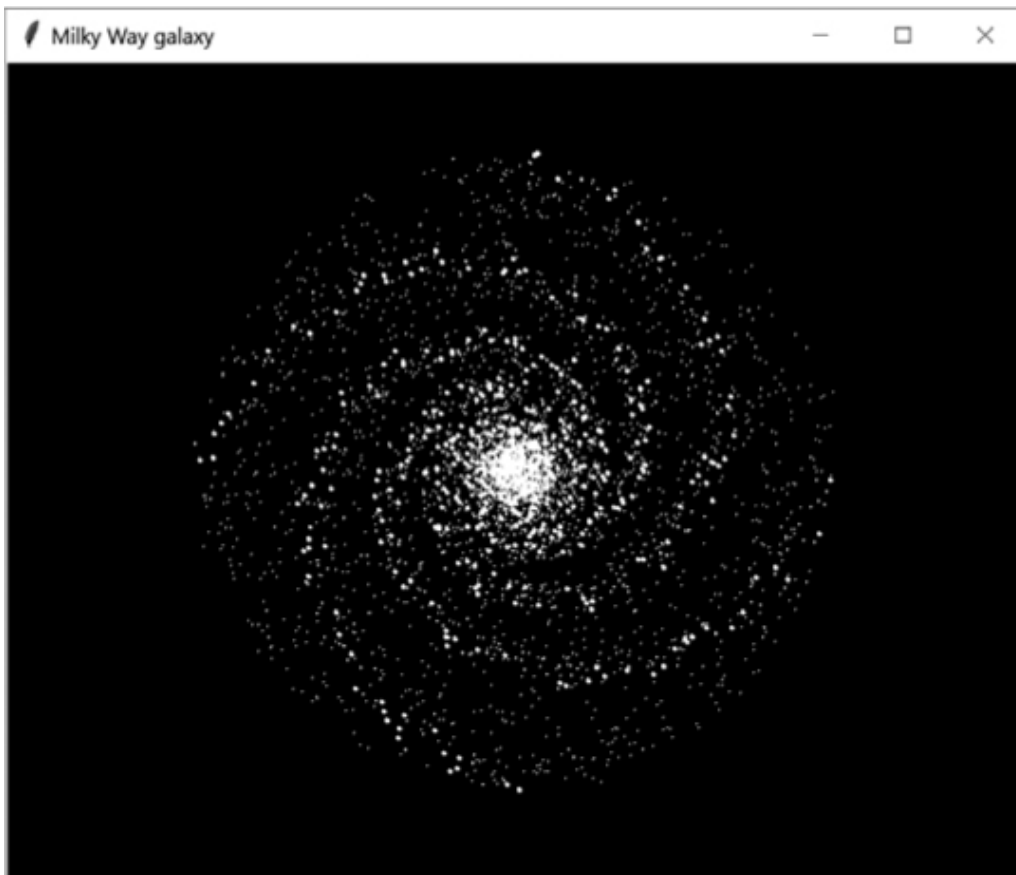


Figure 10-11: Filling out the spiral arms by shifting spirals and randomly changing star positions

Now it's time to build the spiral lines. First, use a range of values to represent θ in the logarithmic spiral equation ❹. A range of about 520 will produce the galaxy in **Figure 10-9**, which has a central “black hole.”

Otherwise, use a range of $(0, 600, 2)$ —or similar—to produce a bright central core fully packed with stars (**Figure 10-12**). You can tinker with these values until you get your preferred result. Loop through the values in `theta` and apply the logarithmic spiral equation, using cosine for the x-value and sine for the y-value. Note that you add the `fuzz` value, multiplied by the fuzz factor, to the result. Append each (x, y) pair to the `spiral_stars` list.



*Figure 10-12: Graphical model without the central black hole (compare to **Figure 10-9**)*

Later, in the `main()` function, you'll specify the `rot_fac` variable, which will move the spiral around the center. After the program builds the four main arms, it will use `rot_fac` to build four new arms, slightly offset from the first four, to produce the band of dim, trailing stars seen to the left of each arc of bright stars in **Figure 10-11**.

Now that you have the list of star locations, start a `for` loop through the (x, y) coordinates ❺. Then use a conditional statement to choose the main, leading arm and locations for which x is even ❻. For these, use the `canvas` widget's `create_oval()` method to create a star object to post. This method's first four arguments define a bounding box into which the oval fits. The larger the number after x and y , the larger the oval. Make the fill white and don't use an outline; the default outline is a thin black line.

If the x -value is odd, make the star a step smaller. And if the `arm` value is 1, the star is in the shifted arm, so make it as small as possible ❼.

The star objects are for visual impact only. Neither their size nor number is to scale. To be realistic, they would be much, much smaller and much more numerous (over 100 billion!).

Scattering Star Haze

The space between the spiral arms isn't devoid of stars, so the next function (**Listing 10-8**) randomly casts points across the galactic model, with no regard for spiral arms. Think of this as the glow you see in photographs of distant galaxies.

galaxy_simulator.py, part 6

```
❶ def star_haze(disc_radius_scaled, density):  
    """Randomly distribute faint tkinter stars in galactic disc.  
  
    disc_radius_scaled = galactic disc radius scaled to radio bubble diameter  
    density = multiplier to vary number of stars posted  
    """  
    ❷ for i in range(0, disc_radius_scaled * density):  
        ❸ x, y = random_polar_coordinates(disc_radius_scaled)  
        ❹ c.create_text(x, y, fill='white', font=('Helvetica', '7'), text='.')
```

Listing 10-8: Defines the `star_haze()` function

Define the `star_haze()` function and pass it two arguments: the scaled disc radius and an integer multiplier that the function will use to increase the base number of random stars ❶. So, if you prefer a thick fog rather than a light haze, increase the value of the density value when you call the function in `main()`.

Start a `for` loop where the maximum range value is equal to the scaled disc radius multiplied by `density` ❷. By using the radius value, you scale the number of stars to the size of the disc being displayed. Then call the `random_polar_coordinates()` function to get an `(x, y)` pair ❸.

End by creating a display object for the canvas using the (x, y) pair ④. Since you've already used the smallest oval size for the stars along and around the spiral, use the `create_text()` method instead of `create_oval()`. With this method, you can use a period to represent a star. The font size parameter will allow you to scale the haze stars until you find something aesthetically pleasing.

Figure 10-13 is a comparison between the galactic model without the star haze (left) and with the star haze (right).

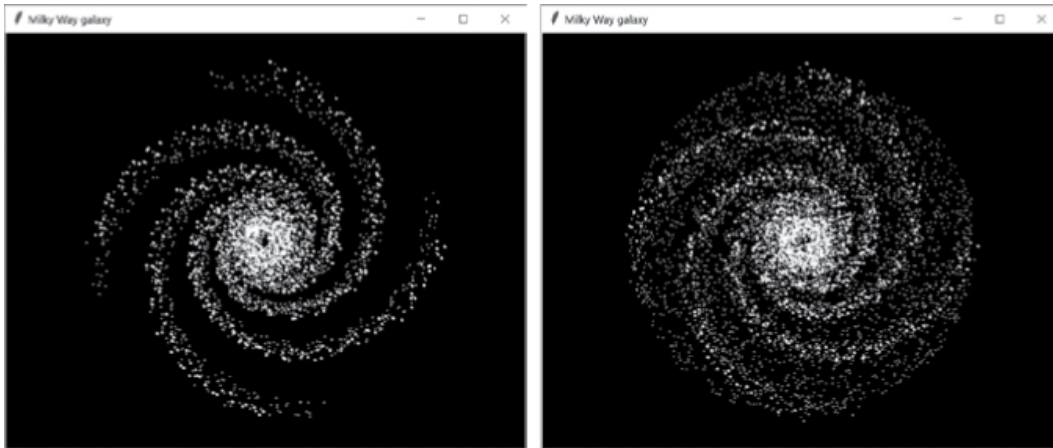


Figure 10-13: Galactic model without star haze (left) versus with star haze (right)

You can be creative with the haze. For example, you can make the stars more numerous and color them gray, or use a loop to vary both their size and color. Don't use green, however, as there are no green stars in the universe!

Defining the `main()` Function

Listing 10-9 defines the `main()` function in `galaxy_simulator.py`. It will make the calls to scale the galaxy, calculate the probability of detection, build the galaxy display, and post statistics. It will also run the `tkinter` main loop.

galaxy_simulator.py, part 8

```
def main():  
    """Calculate detection probability & post galaxy display & statistics."""  
    ① disc_radius_scaled, disc_vol_scaled = scale_galaxy()
```

```

detection_prob = detect_prob(disc_vol_scaled)

# build 4 main spiral arms & 4 trailing arms
❷ spirals(b=-0.3, r=disc_radius_scaled, rot_fac=2, fuz_fac=1.5, arm=0)
    spirals(b=-0.3, r=disc_radius_scaled, rot_fac=1.91, fuz_fac=1.5, arm=1)
    spirals(b=-0.3, r=-disc_radius_scaled, rot_fac=2, fuz_fac=1.5, arm=0)
    spirals(b=-0.3, r=-disc_radius_scaled, rot_fac=-2.09, fuz_fac=1.5, arm=1)
    spirals(b=-0.3, r=-disc_radius_scaled, rot_fac=0.5, fuz_fac=1.5, arm=0)
    spirals(b=-0.3, r=-disc_radius_scaled, rot_fac=0.4, fuz_fac=1.5, arm=1)
    spirals(b=-0.3, r=-disc_radius_scaled, rot_fac=-0.5, fuz_fac=1.5, arm=0)
    spirals(b=-0.3, r=-disc_radius_scaled, rot_fac=-0.6, fuz_fac=1.5, arm=1)
    star_haze(disc_radius_scaled, density=8)

# display legend
❸ c.create_text(-455, -360, fill='white', anchor='w',
               text='One Pixel = {} LY'.format(SCALE))
    c.create_text(-455, -330, fill='white', anchor='w',
               text='Radio Bubble Diameter = {} LY'.format(SCALE))
    c.create_text(-455, -300, fill='white', anchor='w',
               text='Probability of detection for {:,} civilizations = {}'.format(
                   NUM_CIVS, detection_prob))

# post Earth's 225 LY diameter bubble and annotate
❹ if SCALE == 225:
    ❺ c.create_rectangle(115, 75, 116, 76, fill='red', outline='')
        c.create_text(118, 72, fill='red', anchor='w',
                     text="<----- Earth's Radio Bubble")

# run tkinter loop
❻ root.mainloop()
❼ if __name__ == '__main__':
    main()

```

Listing 10-9: Defines and calls the `main()` function

Start `main()` by calling the `scale_galaxy()` function to get the scaled disc volume and radius ❶. Then call the `detect_prob()` function and pass it the `disc_vol_scaled` variable. Assign the results to a variable named `detection_prob`.

Now build the galaxy display (graphical model) ❷. This calls the `spirals()` function multiple times, with small changes to each call. The `arm` parameter designates the bright main arms and the faint trailing arms. The `rot_fac` (rotation factor) variable determines where the spiral plots. The slight change in rotation factor between arms 0 and 1 (for example, 2 to 1.91) is what causes the faint arm to plot slightly offset from the bright arm. Finish the display by calling the `star_haze()` function. Again, feel free to experiment with any of these parameters.

Next, display a legend and statistics. Start with the scale ❸ and radio bubble diameter followed by the probability of detection for the given number of civilizations. Arguments include the x and y coordinates, a fill (text) color, a justification anchor—with left represented by `w` for “west”—and the text. Note the use of `{:,}` to insert a comma as a thousand separator. This is part of the newer *string format method*. You can read more about it at <https://docs.python.org/3/library/string.html#string-formatting>.

If the user has selected a radio bubble diameter of 225 LY ❹, then the display is at the same scale as our own emissions bubble, so post a red pixel at the approximate location of our solar system and annotate it ❺. There are a number of ways to display a single pixel using `tkinter`. Here, you use the `create_rectangle()` method, but you can also make a line that is one pixel long with the following statement:

```
c.create_line(115, 75, 116, 75, fill='red')
```

With the `create_rectangle()` method, the first two arguments are points (`x0`, `y0`), which correspond to the top-left corner, and (`x1`, `y1`), the location of the pixel just outside of the bottom-right corner. With the `create_line()` method, the arguments are for the starting and ending points. The default line width is one pixel.

End the `main()` function by executing the `tkinter mainloop()` function, also known as the *event loop* ❻. This keeps the `root` window open until you close it.

Back in the global space, end the program by allowing it to be run stand-alone or called as a module in another program ⑦.

The final display will look like **Figure 10-14**, shown with the Earth's radio bubble and a central black hole.

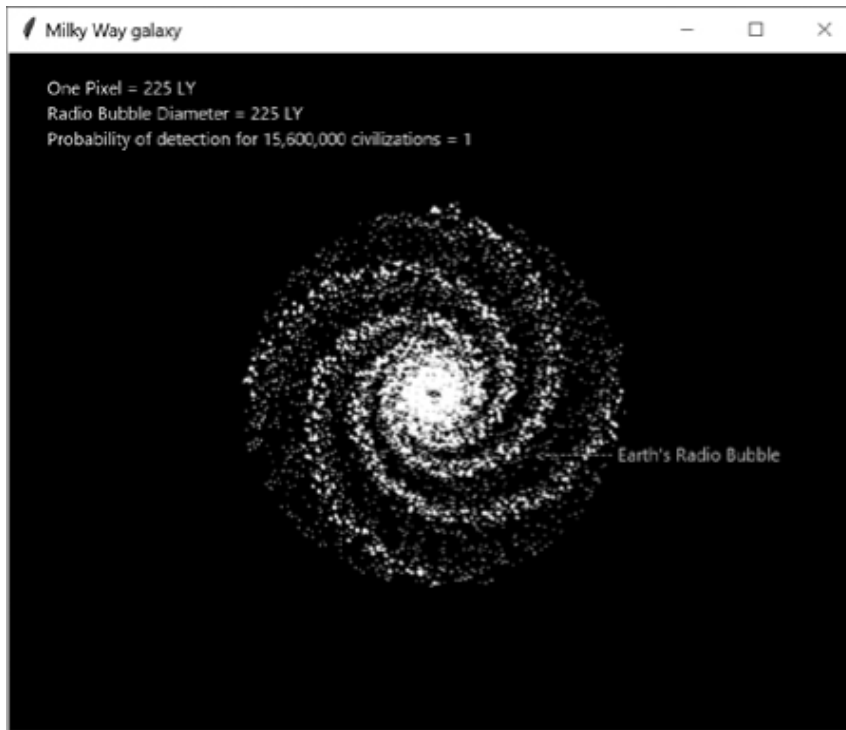


Figure 10-14: The final display, with Earth's 225 LY diameter radio bubble posted on the galactic map

Note that, despite the fact that our radio bubble is no bigger than a pinprick at this scale, if civilizations had a detection range of 112.5 light-years, and if there were as many of these civilizations as predicted by current high-side parameters for the Drake equation, the probability of detection is 1!

Results

Given the enormous uncertainty in the inputs and the use of simplifying assumptions, you're not looking for accuracy here. What you're looking for is *directionality*. Should we (or anyone like us) expect to detect another civilization that isn't actively trying to contact us? Based on **Figure 10-15**, probably not.

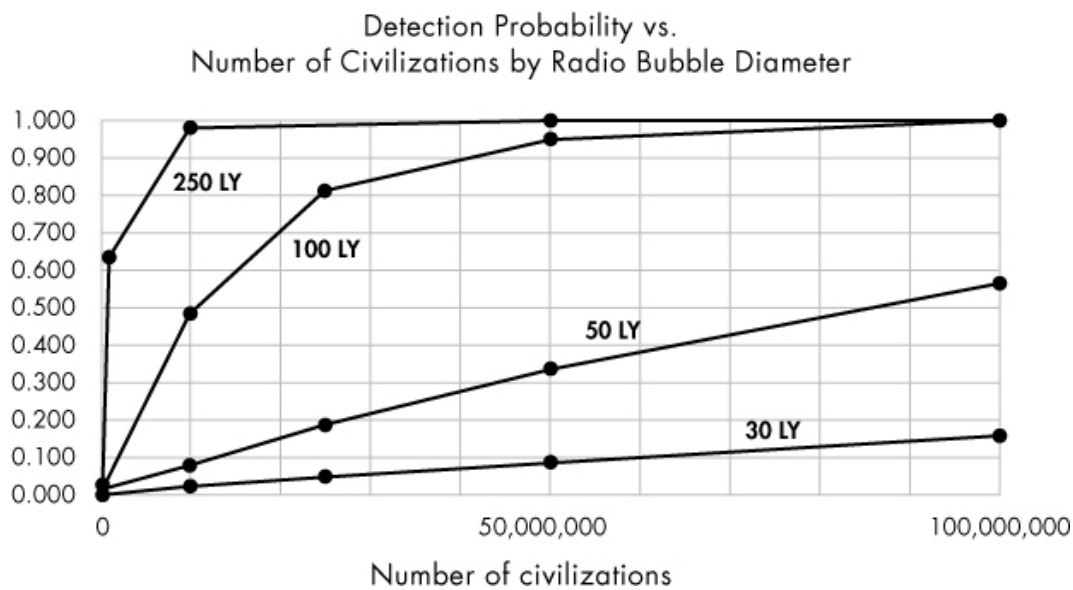


Figure 10-15: Probability of one civilization detecting another for different radio bubble diameters and different numbers of civilizations in the galaxy

With our current technology, we could detect emissions from a civilization as far away as 16 LY, which equates to a 32 LY diameter radio bubble. Even if the galaxy is filled with 15.6 million advanced civilizations, as predicted by the updated Drake equation in the Wikipedia article, the chance of detecting 32 LY radio bubbles is less than 4 percent!

Take another look at **Figure 10-14**, and you can begin to appreciate the sheer enormity and emptiness of our galaxy. Astronomers even have a word for this: *Laniakea*, Hawaiian for “immeasurable heaven.”

Earth is, as Carl Sagan described it, just a “mote of dust, suspended in a sunbeam.” And recent studies suggest that the window of opportunity for detecting civilizations with radio waves is much smaller than we thought. If other civilizations follow our lead and switch to digital signals and satellite communications, then their incidental radio leakage will drop by at least a factor of four. We all become unintentionally stealthy, blooming for a hundred years or so, then fading away.

Given these facts, it’s not surprising that the government no longer funds the search for extraterrestrial intelligence using radio telescopes. These days, efforts are shifting to optical methods that look for signature gases in the atmospheres of exoplanets, such as the waste products of life and industrial activity.

Summary

In this chapter, you gained experience using `tkinter`, `matplotlib`, and `NumPy`. You generated a polynomial expression for making reasonable estimates of the likelihood of detecting incidental alien radio transmissions, and you used the always-available `tkinter` module to add a cool visual component to the analysis.

Further Reading

Are We Alone? Philosophical Implications of the Discovery of Extraterrestrial Life (BasicBooks, 1995) by Paul Davies is a thoughtful look at the search for alien life, told by an eminent scientist and award-winning science writer.

“A New Formula Describing the Scaffold Structure of Spiral Galaxies” (*Monthly Notices of the Royal Astronomical Society*, July 21, 2009) by Harry I. Ringermacher and Lawrence R. Mead (<https://arxiv.org/abs/0908.0892v1>) provides formulas for modeling the shapes of spiral galaxies observed by the Hubble telescope.

“Tkinter 8.5 Reference: A GUI for Python” (New Mexico Tech Computer Center, 2013) by John W. Shipman is a useful supplement to the official `tkinter` docs. It can be found at <http://infohost.nmt.edu/tcc/help/pubs/tkinter/tkinter.pdf>.

Another useful online `tkinter` resource is <https://wiki.python.org/moin/TkInter/>.

Tkinter GUI Application Development HOTSHOT (Packt Publishing, 2013) by Bhaskar Chaudhary uses a project-based approach to teach `tkinter`.

Practice Projects

Try these three spin-off projects. You can find them in the appendix or download them from <https://www.nostarch.com/impracticalpython/>.

A Galaxy Far, Far Away

Tired of living in the Milky Way galaxy? Heck, who isn’t? Fortunately, there’s more in heaven and earth than just logarithmic spirals. Use

Python and `tkinter` to build us a new home—but not necessarily a realistic home. For inspiration, visit online articles like Alexandre Devert’s post on his Marmakoide’s Blog, “Spreading Points on a Disc and on a Sphere” (<http://blog.marmakoide.org/>). The example shown in **Figure 10-16** was built with *galaxy_practice.py*.

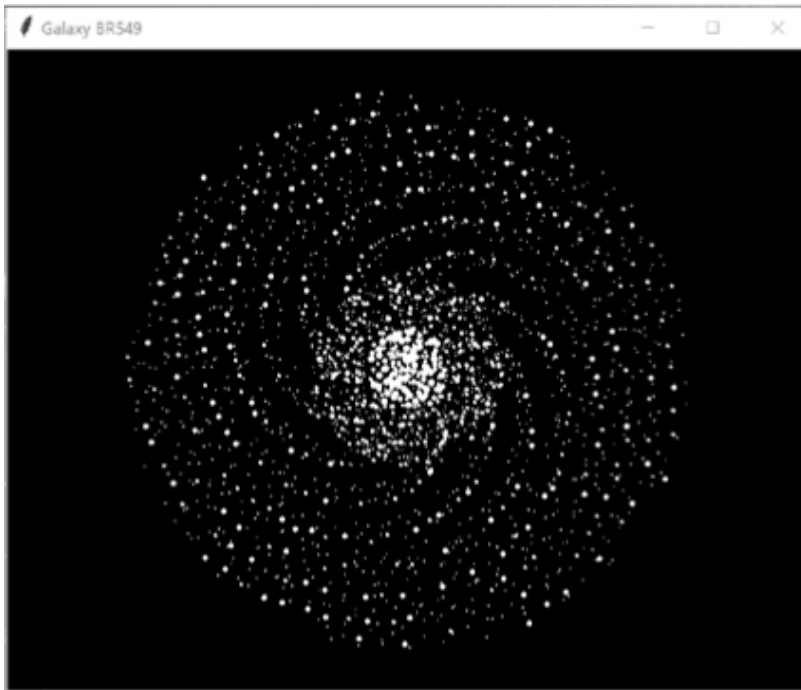


Figure 10-16: Galaxy produced by the `galaxy_practice.py` program

Building a Galactic Empire

Pick a location in the galaxy, an average travel speed of 5 to 10 percent of the speed of light, and a time step of 500,000 years. Then model the expansion of a space-faring empire. At each time step, calculate the size of the expanding *colonization bubble* and update the galaxy map. Check your results by placing the home-world location at the center of the galaxy, setting the speed to 1, and confirming that it takes 50,000 years to reach the edge of the galaxy.

When you have the program up and running, you can perform interesting experiments. For example, you can test how fast we would need to go to explore the galaxy in 10 million years, as mentioned in the introduction to this chapter (see **Figure 10-17**).

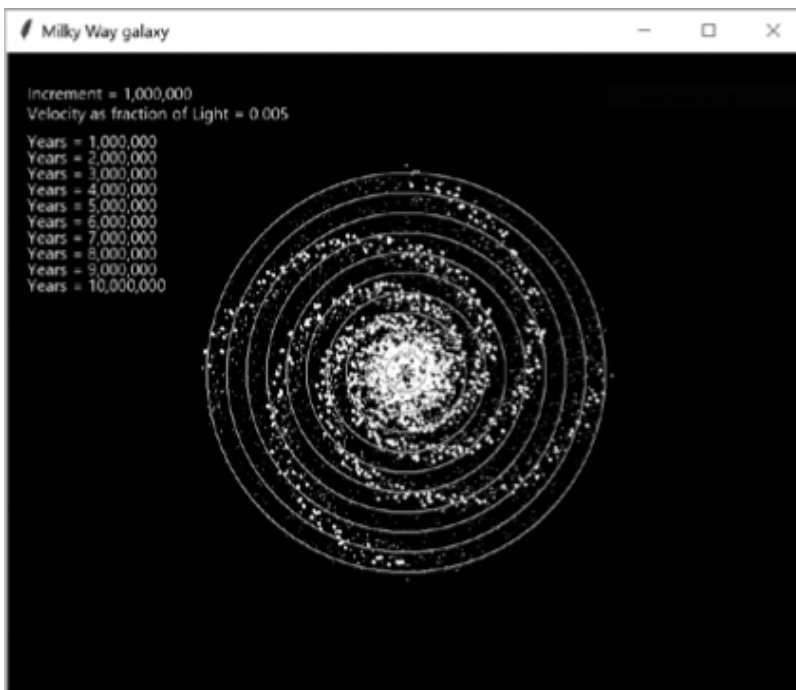


Figure 10-17: A core-located empire's expansion using travel below light speed over 10 million years

You could also estimate how much of the galaxy the *Star Trek* Federation could have explored in its first 100 years, assuming they averaged 100x light speed at warp 4 (**Figure 10-18**).

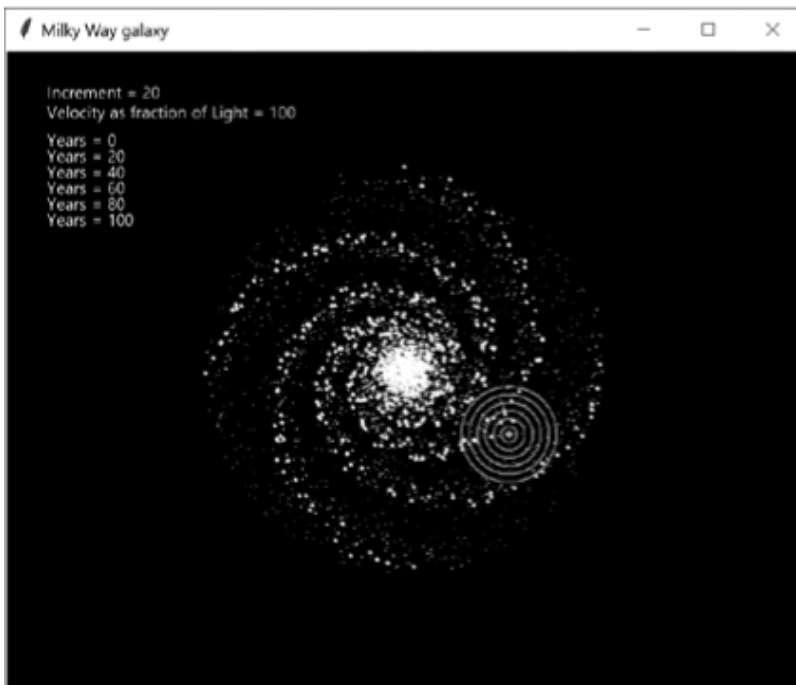


Figure 10-18: Expansion of the Star Trek Federation in first 100 years under warp factor 4

These figures were built with the *empire_practice.py* program.

A Roundabout Way to Predict Detectability

Another way to predict the probability of detection is to use polar coordinates to distribute civilizations—as xyz points—in the galactic disc and then round the points to the nearest radio bubble *radius*. Points that share the same location represent civilizations that could detect each other. But be careful—this method rounds using cubes rather than spheres, so you’ll need to convert the radius to the side of a cube that produces the same volume.

Write the program so that it predicts the probability of detecting 16 LY *radius* bubbles (the limit of our current technology) given 15,600,000 transmitting civilizations randomly distributed throughout the galaxy (updated Drake equation output from Wikipedia). Use the full 50,000 LY radius and 1,000 LY height of the galactic model when distributing the civilizations.

For a solution, see *rounded_detection_practice.py*. Note that the program will take several minutes to run.

Challenge Projects

Here are a few follow-up projects to try on your own. Remember that I don’t provide solutions to challenge projects.

Creating a Barred-Spiral Galaxy

Our understanding of the Milky Way evolves as we obtain and analyze new astronomical data. Scientists now believe that the core of the galaxy is elongated and bar shaped. Use the equations provided in the Ringermacher and Mead paper, cited in “**Further Reading**” on **page 212**, to create a new `tkinter` visual model of the galaxy that honors the barred-spiral concept.

Adding Habitable Zones to Your Galaxy

Solar systems have *Goldilocks zones* that are favorable for the development of life. Planets orbiting in these zones stay warm enough for at least some of their water to remain in liquid state.

There is also a theory that galaxies, like solar systems, have *habitable zones* in which life is more likely to develop. One definition of the habitable zone for the Milky Way places its inner boundary about 13,000 LY from the galactic center and its outer boundary about 33,000 LY from the center (**Figure 10-19**). The core is excluded due to the high levels of radiation, large number of supernovas, and complex orbit-disrupting gravitational fields resulting from all the closely spaced stars. The rim areas are condemned due to low metallicity, which is crucial to the development of planets.

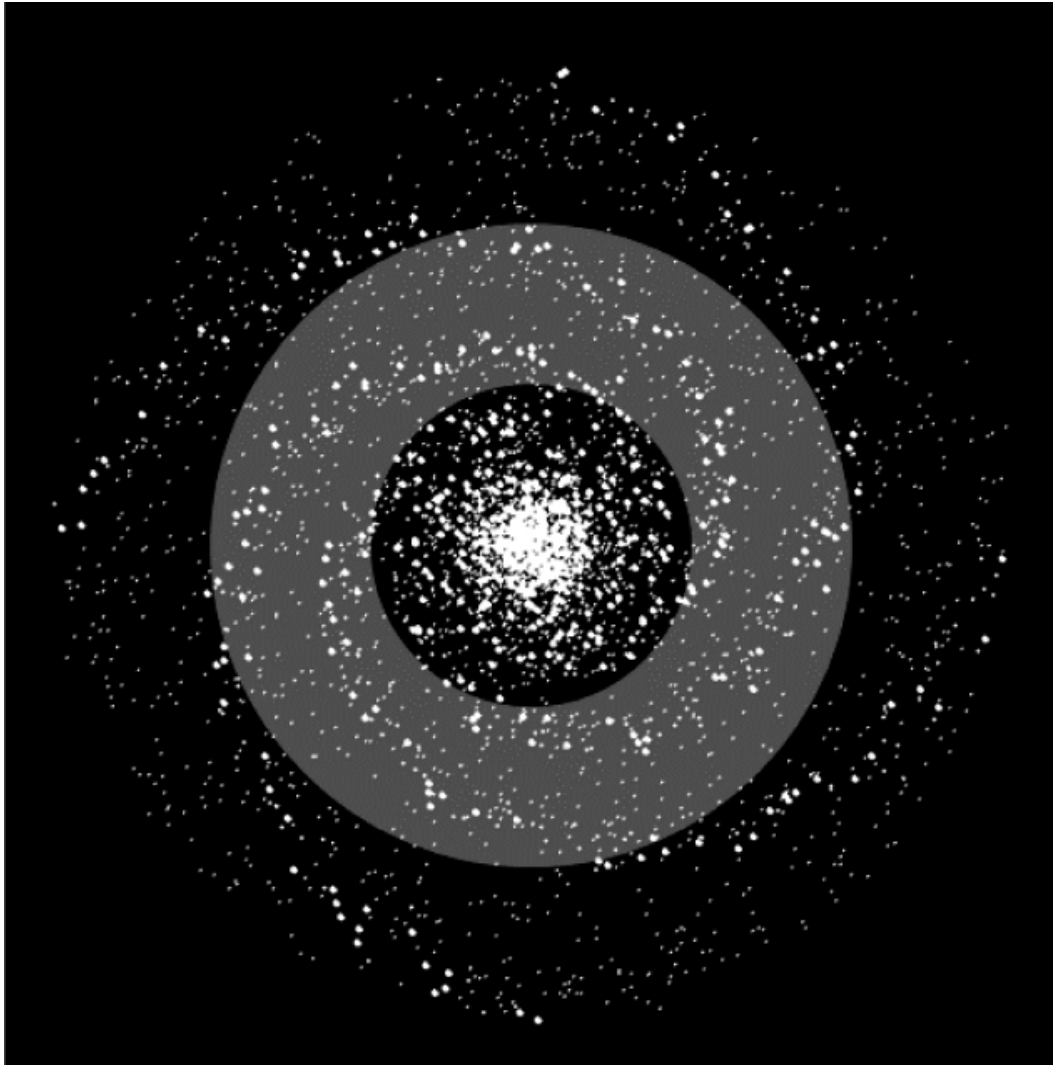


Figure 10-19: Approximate galactic habitable zone (shaded) superimposed on the Milky Way model

A refinement of the habitable zone model excludes spiral arms, for reasons similar to those applied to the core. Our own existence doesn't contradict this. Earth is located in the Orion "spur," a relatively small feature between the Sagittarius and Perseus arms.

Edit the *galaxy_simulation.py* program so that it uses only the volume in the galactic habitable zone, however you define it. You should research what these volumes may be and what effect they'll have on the number of civilizations (N) that the Drake equation calculates. Consider using *regions*, such as the core, spirals, outer rim, and so on, within which N is different but civilizations are still randomly distributed. Highlight these regions on the galactic map and post their probability-of-detection estimates.